

PRACTICAL – 1(A)

Aim: To create and implement a Java coin-operated snack vending program using an enum and switch expression to convert coin types into values, while practicing Scanner, loops, conditional logic, and calculating the change returned after reaching the required price.

Code Screenshot:

```
import java.util.Scanner;

// Enum to store the available coin types.
enum Coin {
    ONE, TWO, FIVE, TEN
}

public class VendingMachine {

    Run main | Debug main
    public static void main(String[] args) {
        int total = 0;
        Scanner sc = new Scanner(System.in);

        // Keep asking for coins until payment is complete.
        while(true) {

            System.out.print("Enter coin (ONE, TWO, FIVE, TEN): ");
            Coin coin;

            // Try to convert user input into a valid coin
            try {
                coin = Coin.valueOf(sc.next().toUpperCase());
            } catch (IllegalArgumentException e) {
                System.out.println(" Not a valid coin, try again.");
                continue;
            }

            int value = switch (coin) {
                case ONE -> 1;
                case TWO -> 2;
                case FIVE -> 5;
                case TEN -> 10;
            };

            total += value;

            if (total < 15 ) {
                System.out.println(" Inserted so far: " + total);
            }
            else {
                System.out.println("Paid. Change: " + (total - 15));
                break;
            }

            // sc.close();
        }
    }
}
```

Output Screenshot:

```
PS C:\Users\Aryan\Documents\Charusat\Sem-3\Java> javac VendingMachine.java
PS C:\Users\Aryan\Documents\Charusat\Sem-3\Java> java VendingMachine
Enter coin (ONE, TWO, FIVE, TEN): one
    Inserted so far: 1
Enter coin (ONE, TWO, FIVE, TEN): five
    Inserted so far: 6
Enter coin (ONE, TWO, FIVE, TEN): Five
    Inserted so far: 11
Enter coin (ONE, TWO, FIVE, TEN): Two
    Inserted so far: 13
Enter coin (ONE, TWO, FIVE, TEN): TEN
Paid. Change: 8
```

Practicle-1(B)

Aim: To create and implement a Java toll collection program using a record Vehicle and switch expressions to calculate toll charges, maintain vehicle type counters, use loops and user input, and determine the vehicle type with the highest count along with the total toll collected.

Code Screenshot:

```
import java.util.Scanner;

public class TollBooth {

    record Vehicle(String number, String type) { }

    Run main | Debug main
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int totalToll = 0;
        int bikes = 0, cars = 0, trucks = 0;

        System.out.println("Enter vehicles. Type 'done' for the number to stop.");

        while (true) {
            System.out.print("Vehicle number (or 'done'): ");
            String number = sc.nextLine();
            if ("done".equals(number)) {
                break;
            }
            System.out.print("Vehicle type(bike/car/truck): ");
            String type = sc.nextLine();
            Vehicle vehicle = new Vehicle(number, type);
            int Toll = switch (type) {
                case "bike" -> 20;
                case "car" -> 50;
                case "truck" -> 150;
                default -> 0;
            };
            switch (type) {
                case "bike" -> bikes++;
                case "car" -> cars++;
                case "truck" -> trucks++;
            }
            System.out.println(" " + number + " (" + type + ") pays " + Toll);
            totalToll += Toll;
        }
        String mostFrequent;
        System.out.println("Total toll collected: " + totalToll);
        if (bikes >= cars && bikes >= trucks) mostFrequent = "bike";
        else if (cars >= bikes && cars >= trucks) mostFrequent = "car";
        else mostFrequent = "truck";
        System.out.println("Most frequent: " + mostFrequent);
        sc.close();
    }
}
```

Output Screenshot:

```

PS C:\Users\Aryan\Documents\Charusat\Sem-3\Java> javac TollBooth.java
PS C:\Users\Aryan\Documents\Charusat\Sem-3\Java> java TollBooth
Enter vehicles. Type 'done' for the number to stop.
Vehicle number (or 'done'): G26
Vehicle type(bike/car/truck): bike
    G26 (bike) pays 20
Vehicle number (or 'done'): G48
Vehicle type(bike/car/truck): car
    G48 (car) pays 50
Vehicle number (or 'done'): G19
Vehicle type(bike/car/truck): truck
    G19 (truck) pays 150
Vehicle number (or 'done'): G28
Vehicle type(bike/car/truck): cr
    G28 (cr) pays 0
Vehicle number (or 'done'): G02
Vehicle type(bike/car/truck): bike
    G02 (bike) pays 20
Vehicle number (or 'done'): G37
Vehicle type(bike/car/truck): truck
    G37 (truck) pays 150
Vehicle number (or 'done'): G23
Vehicle type(bike/car/truck): bike
    G23 (bike) pays 20
Vehicle number (or 'done'): done
Total toll collected: 410
Most frequent: bike

```

- **Question-Answer: -**

1. **What is the role of the JVM, and what file does the javac compiler produce?**

- JVM (Java Virtual Machine) runs Java programs by converting bytecode into machine code.
- The javac compiler converts a .java file into a .class file containing Java bytecode.

2. **How does a switch expression differ from a traditional switch statement?**

- A switch expression directly returns a value and uses -> syntax without break.
- A traditional switch statement only controls program flow and requires break to avoid fall-through.

3. Why is an enum a good choice for a fixed set of menu options, and a record for read-only data?

- Enum is a good choice for a fixed set of menu options because it allows only predefined values.
- Record is good for read-only data because its fields are immutable and cannot be changed after the object is created.

Learning Outcome:

After completing these programs, students will be able to use Java enums and records to model fixed values and read-only data, take user input using the Scanner class, apply switch expressions, loops, and conditional statements to solve problems, perform basic calculations, maintain counters and totals, and develop simple real-world applications such as a vending machine and a toll booth using modern Java features.

GitHub Repositories: -

<https://github.com/d26it116/OOP/tree/159f27c0b1a369ac4529973d98446af6a77befb1/Practicle-1>